# Lab 2-F26 IoT Security

ESE 5180: Wireless, Security, & Scaling

Gradescope PDF Submission Deadline: Oct 26, 2026 11:59 PM

Teaching Team Checkoff Deadline: Oct 28, 2026

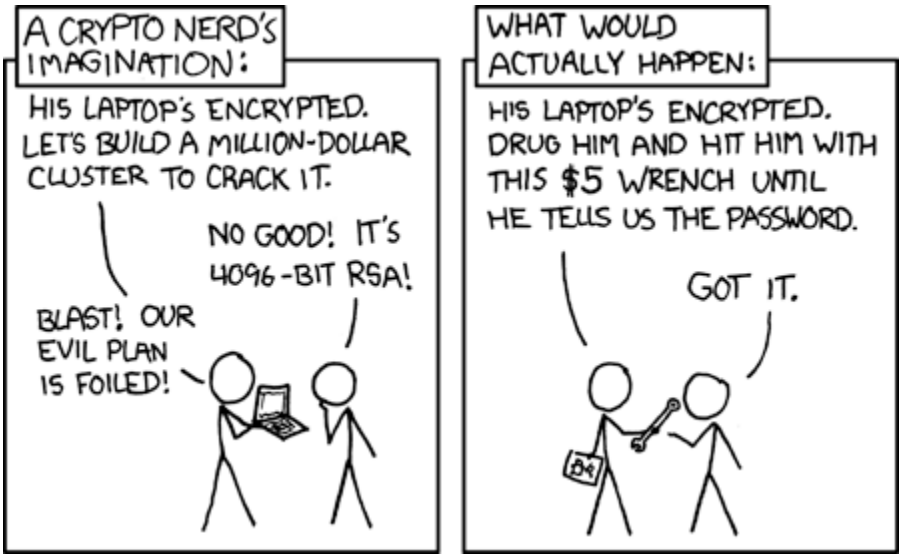
Note: You cannot re-submit/update the Gradescope PDF/GitHub repository after getting the Teaching Team checkoff.

A **group** submission to be submitted on GitHub & Gradescope.

Find your partner here: ESE5180 F26 Lab Group Assignments

**GitHub Repository Template:** https://github.com/ese5180/lab2-iot-security-skeleton

**Late-day requests:** https://forms.gle/YkvTbsuNCR2wdZLN8




xkcd: Security

# Overview

After this lab, you should understand the following:

- How the different security systems are implemented
- How two isolated systems can connect securely in practice
- What kinds of considerations go into picking an appropriate security algorithm

**All the required questions are highlighted in green.**

**Important notes are highlighted in yellow.**

Text highlighted by purple are "think" questions. They are intended to test your understanding of theoretical concepts or guide you in implementation of the lab.

Completing these think questions/prompts is highly recommended - it's common to see these concepts show up in the lab checkoff or quiz questions.

The Gradescope/GitHub submission+demo will account for **~60%** of this lab.
The individual firmware explanations and lab checkoff questions will account for **~40%** of the lab.

# Required Equipment

- **Hardware**
  - Personal laptop computer (Windows, macOS, or Linux)
  - Two NUCLEO-WL55JC2 boards, one for the Client and one for the Server (https://www.digikey.com/short/qtj7bjbm)
  - Micro-USB cables that carry **data**.
  - nRF7002 DK
- **Software**
  - Zephyr RTOS, the Zephyr SDK and the `west` tool
  - STM32CubeProgrammer (used by `west flash` to program the board)
  - A serial terminal (for example `screen`, `minicom` or PuTTY)
  - The starter code from the Lab Dependency repository (see Get the starter files below)

# Getting Started From Scratch

Work through this section once. By the end, you will have built the Client and Server starters and flashed one onto each board.

## Install the toolchain

If needed, follow the official *Zephyr Getting Started Guide*:
https://docs.zephyrproject.org/latest/develop/getting_started/index.html

This lab was developed and checked with Zephyr 4.4 (a 4.4 or newer checkout), Zephyr SDK 1.0.1, `west` 1.5 and Python 3.12. Install the Zephyr SDK and Python virtual environment as the guide describes, and install STM32CubeProgrammer so that `west flash` can program the board.

Each time you open a new terminal, go to your workspace and activate your environment first. For example, if your workspace is `zephyrproject` in your home folder:

| **OS** | **Commands** |
|---|---|
| Linux, macOS | `cd ~/zephyrproject`<br>`source .venv/bin/activate` |
| Windows (cmd) | `cd %USERPROFILE%\zephyrproject`<br>`.venv\Scripts\activate.bat` |
| Windows (PowerShell) | `cd $HOME\zephyrproject`<br>`.venv\Scripts\Activate.ps1` |

`west build` and `west flash` only work **inside the workspace**. If you see `west: unknown command "build"`, you are in the wrong folder. Run the `cd` above and try again.

## Get the starter files

The starter code is in the Lab Dependency repository: https://github.com/ese5180/ese5180deps
The Arduino Crypto library is included in the repository as a git submodule, so clone with submodules:

```
git clone --recurse-submodules https://github.com/ese5180/ese5180deps.git
```

If you already cloned without `--recurse-submodules`, run `git submodule update --init` inside the repository.

Check that `lab2_security/Crypto/src` lists files such as `AES128.cpp` before you continue. If it is empty, the copy in the next section will be empty too and the build fails with "Cannot find source file .../Crypto/src/AES128.cpp".

Inside the `lab2_security` folder, you will use:

```
LoRa_Encrypted_Client_Zephyr/     (code for board 1)
LoRa_Encrypted_Server_Zephyr/     (code for board 2)
Crypto/                           (the Arduino Crypto library, a submodule)
```

Each starter folder contains:

| **File or folder** | **What it is** |
|---|---|
| `src/main.c` | Your code goes here: the radio setup, LEDs, helper functions, IoT and the pairing state machine with `TODO`s. |
| `src/sec_crypto.h` | The crypto functions you call. |
| `src/sec_crypto.cpp` | Wrapper that connects `sec_crypto.h` to the crypto library. You should not need to edit it. |
| `shim/` | A small file that connects the library's random numbers to the board's hardware random number generator. |
| `CMakeLists.txt` | Builds your code together with the library in `Crypto/`. You should not need to edit it. |
| `prj.conf` | The Zephyr settings. **You must add to this file** (next section). |

## Set up your projects

**Work in a path without spaces.** Zephyr cannot build an app whose folder path contains a space. Copy the **whole** `lab2_iot_security` folder (so `Crypto/` stays next to the two starter folders) to a short path with no spaces. Use your file manager (Explorer, Finder, or Files), or a command run from the folder where you cloned the repository:

| **OS** | **Command** |
|---|---|
| Linux, macOS | `cp -r "ese5180deps/lab2_security" ~/lab2_security` |
| Windows (PowerShell) | `Copy-Item -Recurse "ese5180deps \lab2_security" C:\lab2_security` |

Then work in the new folder (`~/lab2_security`, or `C:\lab2_security` on Windows). On Windows, avoid a home folder whose name contains a space. Commit your finished work to your own group repository.

**Check the copy:** the `Crypto/src` folder in your copy must list `AES128.cpp` and the other library files. If it is empty, go back to the repository, run `git submodule update --init "lab2_security/Crypto"`, and copy again.

**Note:** Workbench for Zephyr — Zephyr Project Documentation is a great extension that you might enjoy for Zephyr application development.

The starters already build the crypto library for you, but the base `prj.conf` only has the logging and LoRa settings. **Edit `prj.conf` in both starter folders and add the following:**

| Setting | Why |
|---|---|
| `CONFIG_CPP=y`, `CONFIG_STD_CPP17=y` | The crypto library is C++. |
| `CONFIG_ENTROPY_GENERATOR=y` | Turns on the board's hardware random number source. |
| `CONFIG_CSPRNG_NEEDED=y` | Makes the cryptographically secure random generator available (the shim uses it to make keys). |
| `CONFIG_MAIN_STACK_SIZE=8192` | Ed25519 uses more stack than the default provides. |

If a build fails, read the first error message. For example, an `undefined reference` error that mentions the random number generator (`sys_csrand_get`) means `CONFIG_ENTROPY_GENERATOR` or `CONFIG_CSPRNG_NEEDED` is missing, and one that mentions `operator delete` means `CONFIG_CPP` is missing. The Zephyr documentation (https://docs.zephyrproject.org) describes Kconfig in detail.

## Build and flash

Go to your Zephyr workspace and activate the environment (Section above), then build the Client and flash it with the board connected over USB:

```
cd ~/zephyrproject
west build --pristine -b nucleo_wl55jc
~/lab2_security/LoRa_Encrypted_Client_Zephyr
west flash
```

Then do the same for the Server on the second board:

```
cd ~/zephyrproject
```

```
west build --pristine -b nucleo_wl55jc
~/lab2_security/LoRa_Encrypted_Server_Zephyr
west flash
```

On Windows, use your own paths (forward slashes work), for example:

```
west build --pristine -b nucleo_wl55jc
C:/lab2_security/LoRa_Encrypted_Client_Zephyr
```

Both builds use the same default `build` folder, so build and flash one board, then build and flash the other. Do not build both and then flash. If `west flash` picks the wrong board, connect one board at a time while flashing.

## Open the serial console

The board's ST-LINK appears as a virtual serial port. Use **115200 baud**:

| **OS** | **How** |
|---|---|
| Linux | `screen /dev/ttyACM0 115200` |
| macOS | `screen /dev/tty.usbmodem* 115200` |
| Windows | PuTTY (or another terminal) on the `COM` port shown in Device Manager, 115200 baud |

To leave the `screen`, press `Ctrl-A` then `K`, then `y`. Press the black reset button on the board to see the boot messages again. If you only have one data cable, watch the console of one board and power the other from any USB power source (a phone charger works).

**What a good first run looks like:** The unmodified starter builds and flashes, then prints the airtime line and `Error` and stops (the Server keeps printing `Error`). That is expected: the state machine is empty until you fill in the `TODO`s in `src/main.c`. It means your setup works.

# 1. Implementing Our Own LoRa Security

In Lab 1 Wireless Comms, your LoRa board blasted unencrypted messages to all receivers. We had a small stopgap solution using Team names as IDs, but we can do much better using the security systems we have covered.

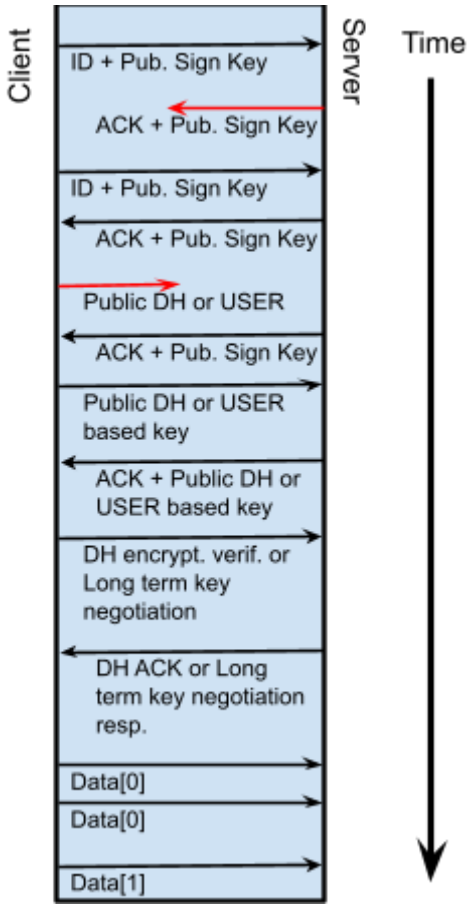
Your task is to upgrade the LoRa-based communication. We've added two files for you to use in our Lab Dependency repository: "LoRa_Encrypted_Client_Zephyr" and "LoRa_Encrypted_Server_Zephyr."

You must implement four stages for both the Client and the Server systems.

1. **Initial Contact Verification**
    a. Here, your system must **identify the correct device** to connect to using **packet advertisements**.
        i. You'll have to define how you want to verify this and why other similar devices can't be mistaken for it. Some ideas are:
            1. Unique IDs
            2. Unique packet structures
        ii. You will also need to have a response packet to initiate the connection.
    b. In the same ID verification packet (and response), they must exchange some kind of credentials to verify future messages (i.e., hash signature keys)
2. **Initial Key exchange**
    a. Here, your system must securely exchange information to obtain a shared encryption key
    b. Key sharing must be one of the following options:
        **i. User-based authentication**
            1. The user must type in a 16-byte key or a secure passphrase that can be hashed into a key
            2. Note: You'll have to pick how the key is chosen, but NO keys should be sent wirelessly here - only a user can transfer this code.
        **ii. Diffie-Hellman Key Exchange**
3. **Long-term key confirmation**
    a. Your system must use the initial keys sent to securely confirm a long-term key for use.
        i. If you used User-based authentication, you will utilize the user-based key to communicate over a new encryption key that both sides will use.
        ii. If you used the Diffie-Hellman Key Exchange, you'll need to confirm that there is a packet with the correct encryption and signature to proceed.
4. **Encrypted Data Sending**
    a. Your system must use the secure encryption key to send and receive encrypted messages.
    b. You will implement a UDP message where packet losses are not confirmed, but a single message may be sent multiple times (in our case, at least 2 times).

   i. Note: Your receiver will then need a way to ignore duplicate messages
   ii. Extra Credit: Implement a TCP-like system where every message is guaranteed to be received through an acknowledgment response
3. This stage must come last! You can't send encrypted messages until you have a valid and secure key.
4. Note: You'll have to do this in a way such that you can guarantee that the other device has received the correct data.

Consider the figure to the right to help visually represent how this could work. This is a transaction diagram showing an example of a full-key handshake.




Note:

- You can choose the exact format for each of these message types
- The Server must send the correct ACK packet to let the client know it must move on and to relay any data the client will need
    - Also, notice that the Server doesn't know if the Client receives anything, so it must keep resending the ACK message.

For the purposes of the purposes of this lab, use the following for these processes:

- Diffie-Hellman: <Curve25519.h>
- Hash Signatures: <Ed25519.h>
- Hashing: <SHA256.h>
- Encryption: <AES.h>
    - Note: for our purposes, we still use the AES128 object - later, we will consider the tinyAES128 and smallAES128 versions.

## Addendum: Block Ciphers

Block ciphers leverage the idea of encrypting entire units of data instead of encrypting character by character. In this class, we won't be able to get into the details (take CIS Cryptography classes if you're interested!), but for this class, it's enough to know that block ciphers must take in a specific size of input and release a specific size of output. For AES128, the input and output must be 128 bits (16 bytes). For messages that are shorter, you can just fill in with the null character or some other specified character like “0”s.

You can double-check the size with the built-in “blockSize() ” function. (Note: some implementations will automatically chunk up larger inputs. Our library does, but you still need at least 16 bytes in the input.)

**Questions to Guide Your Understanding**

- For our given board, how many bytes do you have for your LoRa packets?
    - In this case, consider the given constants in the starter code for your answer - these will be tuned to the expected hardware limitations. Do not count the header that the system automatically attaches to your message (also in the constants).
- Given that you will need a 32-byte signature in your message, how large can your messages be (assuming you don’t add any other packet information)?
- Consider the relatively extreme range of LoRa systems. How does this introduce vulnerabilities into your system?

After you implement your design, answer the following in your README:

(1.1) What key-sharing method did you choose? What considerations (ease of implementation, security, etc) went into picking this system?

- What vulnerabilities would this introduce into your system? Why might this not matter for a LoRa device?
- Given this, argue why your system still satisfies every element of the CIA requirements.

# Requirements

To summarize, your explicit requirements are as follows:

- Security requirements:

| Req - ID | Requirement |
|---|---|
| SEC - 1 | All packets must have a hash signature to verify identity (with the first advertising packet and response having the public key for verification). |
| SEC - 2 | There shall be an initial key exchange that is "secure" based on CIA principles. |
| SEC - 3 | There shall be a system to verify the key transfer and establish a long-term key for use. |
| SEC - 4 | Data sent after the handshake shall be encrypted using a secure key and follow the CIA principles. |

- Networking requirements

| Req - ID | Requirement |
|---|---|
| NET- 1 | Packet structures for each stage must be clearly defined (you can reuse structures with similar purposes for different stages) |
| NET - 2 | During the handshake, if either side gets no response in 1 minute, it should reset to the start |
| NET - 3 | During data transfer, each message should be sent more than once, and the receiver should ignore duplicate packets (these packets can be used to count as a response for the purposes of NET-2) |
| NET - 4 | Data from the Client side should be transferred to the Server side and printed to the console. |

Commit your "LoRa_Encrypted_Client" and "LoRa_Encrypted_Server" implementations to the GitHub repository.
Do not commit the `build_*` folders - see this .gitignore template from Zephyr to simplify your commits: zephyr/.gitignore at main · zephyrproject-rtos/zephyr

You will be expected to demonstrate your firmware handling all of the requirements during the Teaching Team Checkoff.

# 2. Encryption Speed and Memory Tradeoffs

AES can be implemented in several ways, and the choice affects how fast it runs and how much memory it takes. On a microcontroller with limited RAM and flash, both matter. In this section you will measure the same AES-128 implemented four ways on the NUCLEO-WL55JC and compare them.

## Background

Fast software AES precomputes parts of each round into lookup tables (about 8.7 KB in total), so each round becomes a few table reads and XORs instead of heavy arithmetic. Where those tables are stored is a design decision:

| **Variant** | **Overlay file** | **Implementation** |
|---|---|---|
| SW, RAM tables | `sw_ram.conf` | Software AES (Zephyr PSA Crypto). Tables are generated at boot and stored in RAM. |
| SW, ROM tables | `sw_rom.conf` | Same code. Tables are precomputed and stored in flash. |
| SW, ROM + fewer | `sw_rom_fewer.conf` | Only one of every four tables is stored in flash. The others are produced at runtime with bit rotations. |
| HW | `hw.conf` | The STM32WL55 AES peripheral. No tables. |

All four produce identical ciphertext. Only speed and memory differ.

## Setup

The starter code is in `lab2_security/AES_Benchmark_Zephyr/`. It runs on a single board and does not use the radio. The variant is selected at build time with an overlay file, so the code does not change between variants.

From your Zephyr workspace, with the virtual environment active:

```
APP=<path to>/lab2_iot_security/AES_Benchmark_Zephyr
west build -p -b nucleo_wl55jc -d $APP/build_sw_ram $APP -- -DEXTRA_CONF_FILE=sw_ram.conf
west flash -d $APP/build_sw_ram
```

If you do not have STM32CubeProgrammer installed, add `--runner openocd` to the flash command.

Open a serial console at 115200 baud and press the reset button on the board. The line `FIPS-197 known-answer test: PASS` confirms the AES implementation is correct before anything is timed.

## Task 1: Timing code

Open `src/main.c` and complete the section marked **YOUR TASK**. For each variant the program should measure:

- **Key setup:** the time to prepare a new key
- **Encrypt:** the time to encrypt one 176-byte message
- **Decrypt:** the time to decrypt it back

Use Zephyr's cycle counter (`timing_counter_get()`) rather than a millisecond timer. A single AES block takes microseconds, so a millisecond timer would read zero. Repeat each measurement `ITERATIONS` times and keep anything you are not measuring outside the timed window.

Once complete, the board prints the results along with a `CSV,...` line.

## Task 2: Measure all four variants

Build and flash each overlay (`sw_ram`, `sw_rom`, `sw_rom_fewer`, `hw`), using a separate build folder for each, for example `-d $APP/build_sw_rom`. Save the serial output from each run.

## Questions

**(2.1) Speed.** Fill in from your serial output:

| **Variant** | **Key setup, first call (cyc)** | **Key setup (cyc)** | **Encrypt (cyc/B)** | **Decrypt (cyc/B)** | **Encrypt 176 B (µs)** |
|---|---|---|---|---|---|
| SW, RAM tables | | | | | |
| SW, ROM tables | | | | | |
| SW, ROM + fewer | | | | | |

| Variant | Key setup, first call (cyc) | Key setup (cyc) | Encrypt (cyc/B) | Decrypt (cyc/B) | Encrypt 176 B (µs) |
|---|---|---|---|---|---|
| HW | | | | | |

**(2.2) Memory.** For each build, run:

```
arm-zephyr-eabi-size $APP/build_<variant>/zephyr/zephyr.elf
```

`text` and `data` are stored in flash. `data` and `bss` occupy RAM.

| Variant | text | data | bss | Flash (text + data) | RAM (data + bss) |
|---|---|---|---|---|---|
| SW, RAM tables | | | | | |
| SW, ROM tables | | | | | |
| SW, ROM + fewer | | | | | |
| HW | | | | | |

**(2.3) Locating the tables.** On this chip, flash starts at `0x08000000` and RAM starts at `0x20000000`. For each build, run:

```
arm-zephyr-eabi-nm -S $APP/build_<variant>/zephyr/zephyr.elf | grep -E " (FT0|FT1|RT0|FSb)$"
```

Write "missing" if a symbol is not listed.

| Variant | FT0 address | FT0 in RAM or flash | FT1 present | FSb address |
|---|---|---|---|---|
| SW, RAM tables | | | | |
| SW, ROM tables | | | | |
| SW, ROM + fewer | | | | |
| HW | | | | |

**(2.4) Choosing a variant.** A coin-cell LoRa sensor encrypts one reading every 10 minutes and has little free RAM. Which variant would you use? Justify your choice in three or four sentences, citing at least two of your measurements.

## Deliverables

Commit the completed `AES_Benchmark_Zephyr` folder and your answers to (2.1) through (2.4) to your group repository.
Do not commit the `build_*` folders - see this .gitignore template from Zephyr to simplify your commits: zephyr/.gitignore at main · zephyrproject-rtos/zephyr

Be prepared to explain your results during checkoff.

## Extra Credit

The hardware variant currently sends one 16-byte block per driver call. Modify `aes_hw_stm32.c` to encrypt the full 176-byte message in a single call using CBC or CTR mode, then report the new cycles per byte and explain the change.

# 3. Leveraging Hardware Unique Keys (HUKs)

Resources for this section
Flash — Zephyr Project Documentation
Non-Volatile Storage (NVS) — Zephyr Project Documentation
9.6. Key management functions — PSA Certified Crypto API 1.1

Though we avoid storing sensitive information on our devices as much as possible, it must be done sometimes. In this section, you'll learn how to prevent the extraction of keys if the MCU firmware is dumped.

- **Objective:** Prevent extraction or cloning of sensitive device credentials (in our case, your LoRaWAN AppKey) stored in internal flash memory.
- **Threat Model:** An attacker physically gains access to the board and extracts the raw Non-Volatile Storage (NVS) flash dump.
- **Security Primitive:** Derive an encryption key at runtime by mixing a static application salt with the STM32WL55's 96-bit Factory Unique Device ID (UID_BASE at 0x1FFF7580). The derived key encrypts sensitive assets before writing them to the Zephyr NVS partition.
- **Verification:** Reading the raw flash dump reveals high-entropy ciphertext. Flashing the exact binary and NVS image onto a second Nucleo-WL55JC board results in a failed decryption check because the UID's will differ.

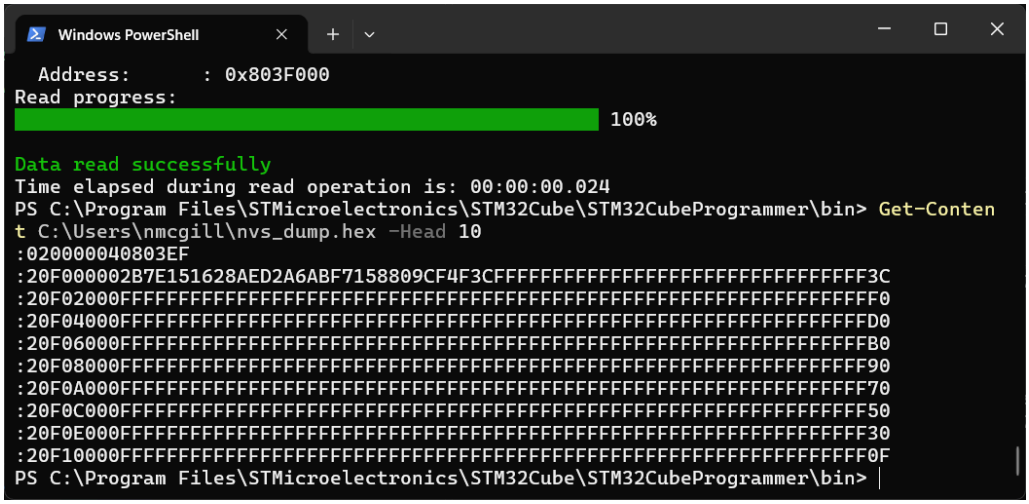
Find the starter code in: ese5180deps/lab2_security/Hardware_Unique_Keys at main · ese5180/ese5180deps

## 3.1 Storing Information in Plaintext

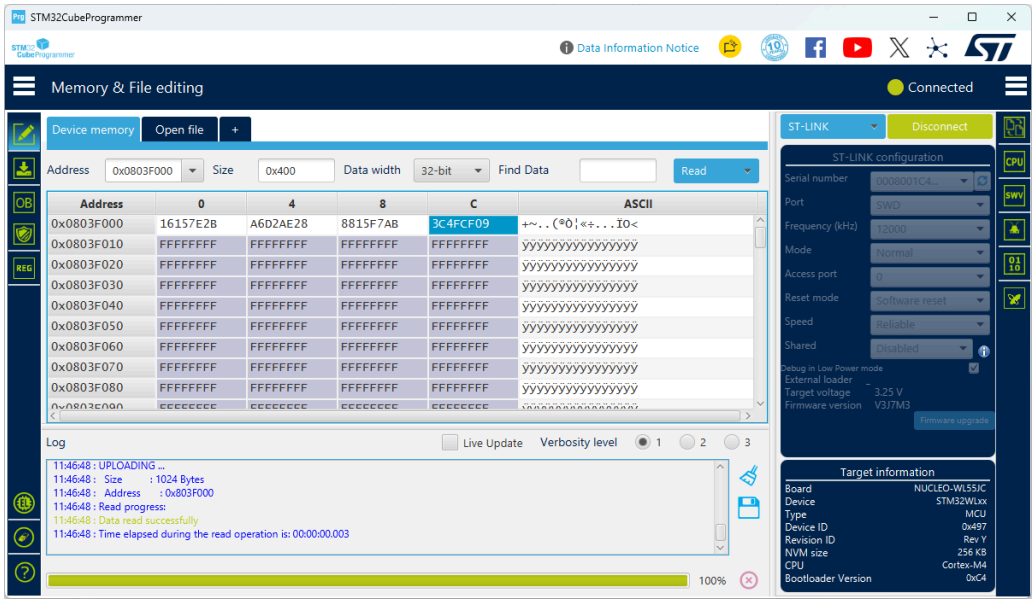
1. Retrieve your 16-byte LoRaWAN AppKey used in the Lab 1 LoRaWAN assignment.
   Write it directly, without encryption, into Zephyr's NVS flash partition.

2. Use STM32 Programmer CLI or GUI to read address 0x0803F000 (NVS partition) and locate your key in plain hex using a hex editor:

```
cd "C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin"
.\STM32_Programmer_CLI.exe -c port=SWD -r 0x0803F000 0x1000 C:\Users\USERNAME\nvs_dump.hex
Get-Content C:\Users\USERNAME\nvs_dump.hex -Head 10
```




Or, the STM32 Programmer GUI window.




(3.1) Submit a screenshot of your **plaintext** key in memory - either in a hex editor if dumped by the CLI, or in the GUI window.

> If the STM32 GUI is in **Data Width** of 32-bit, it can seemingly re-order the bytes in the viewing window. Change it to **8-bit** and see how it matches the 8-bit key array in your firmware.

## 3.2 Hardware Key Derivation

Now, let's encrypt the LoRaWAN key. We'll use a **Key Derivation Function (KDF)** that leverages the Nucleo WL55JC's silicon hardware UID (UID_BASE) and the PSA Crypto API (psa_hkdf / psa_cipher_encrypt). Now, if the firmware binary is dumped, the key's plaintext form will not show up - only its encrypted form.

Continue developing in the same starter project as the previous section. You'll notice more TODOs to implement throughout the code.

(3.2) Submit a screenshot of your **encrypted** key in memory - either in a hex editor if dumped by the CLI, or in the GUI window.

Commit your code to your team's repository.
Do not commit the `build_*` folders - see this .gitignore template from Zephyr to simplify your commits: zephyr/.gitignore at main · zephyrproject-rtos/zephyr

## 3.3 Cloning Attack Simulation

In a cloning attack, a bad actor might pull the contents of the MCU's flash on an official product, then program an unofficial device with the same firmware. This would be an enormous security threat.

In this section, you'll export the entire internal flash image (flash_dump.bin) from Board A and flash it onto Board B. The encrypted LoRaWAN key is stored within the flash image - but, it was encrypted using Board A's UID. Board B will have a different UID and will fail to decrypt the LoRaWAN key.

Observe that Board B's firmware fails authentication/decryption on startup, neutralizing the cloned device. Be prepared to demonstrate this at the lab checkoff.

# 4. AI Disclosure

(4.1) AI tooling can be incredibly helpful in understanding and developing firmware. Describe how you leveraged AI tools within this assignment - there is no penalty for using AI tools. We are mostly curious about better ways to use these tools and improve the assignment.

# 5. Teaching Team Checkoff

Before requesting a Teaching Team checkoff, you must have submitted as a group to GitHub and the **Lab 2 - IoT Security** Gradescope assignment!

Individual team members must submit the name of the Teaching Team member doing the checkoff to this Gradescope assignment: **Lab 2 - IoT Security - Lab Checkoff Questions**.

Find a time to request a Teaching Team Checkoff. Office hours are available on the course website here: https://ese5180.seas.upenn.edu/#officehours

Following the demonstration, each lab partner will be asked to explain in detail how a particular section of the firmware functions. Lab partners may not assist each other for these questions; they must be answered individually.

After this, each lab partner will be asked 2x questions, each. These questions relate to module concepts and implementation within the lab. Lab partners may not assist each other for these questions; they're meant to be answered individually.

It's a good idea to review the lab and relevant lectures before the TA checkoff!

## Demonstration Details

(T1) Meet with a teaching team member for a demo + quick explanation of:

1. Implementing Our Own LoRa Security

Show/describe:

- How are all of the requirements from this section were implemented in your design

(T2) Meet with a teaching team member for a demo + quick explanation of:

2. Encryption Speed and Memory Tradeoffs

Show/describe:

- The results of the testing, and why you might choose one method over another

(T3) Meet with a teaching team member for a demo + quick explanation of:

3. Leveraging Hardware Unique Keys (HUKs)

Show/describe:

- The stored LoRaWAN key being successfully decrypted on Board A.
- Dumping the firmware from Board A, flashing to Board B, and showing the LoRaWAN key retrieval failure.

# Rubric

While the rubric attempts to capture all assignment details, points assigned may vary based on submission quality and teaching team review. Please read the assignment carefully to avoid missing details and losing points. Poor readability/formatting can lose you points on any assignment.

For all questions, 0 points will be awarded if the submission is non-existent, poorly done, or doesn't compile (for firmware assignments).

Group assignments will have a feedback mechanism to help understand the distribution of work and collaboration. If there is a large disparity in work, grades may differ between students in the same group.